Functions

March 6, 2024

A C program consists of many small functions

- May reside in one or more source files, compiled separately and loaded together, along with previously compiled functions from libraries.

Functions

- in C are equivalent to subroutines or functions in Fortran or procedures in Pascal.
- break large computing tasks into smaller, manageable pieces.
- encapsulate computation, thereby hiding details of operations from parts of the program that don't need to know about them.
- enable collaboration and building on existing work.
- are designed to be efficient and easy to use.
- facilitate modularity and code reuse.

Functions: terminology

- Function declaration: announces the existence of the function, with types of parameters and return value
- Function definition: Specifies the function with its body
- Function call: invokes the function
- parameter: a variable named in the parenthesized list in a function definition.
- argument: a variable named in the parenthesized list in a function call.
- The terms FORMAL ARGUMENT and *actual argument* are sometimes used for PARAMETER and *argument*.

Function Declaration

- is a statement that specifies the function's name, return type, and parameters (if any).
- doesn't provide the actual implementation of the function.
- is normally used in header files or at the beginning of a source file to make the compiler aware of the function, which will be defined elsewhere in the code.
- is like a promise to the compiler that a function with a particular signature will be available.

Function Definition

- A program is a set of variable and function definitions. One of the functions has to be main.
- A function definition provides the actual implementation of the function.
- It includes the function's code, specifying what the function does when called.
- You can separate the declaration and definition of a function. This is useful for modularizing code, especially when functions are declared in header files and defined in source files. It allows different parts of a program to be compiled independently.
- A function is defined only once in a program. It can be declared multiple times.

Function Definition: power(m,n)

```
#include <stdio.h>

int power(int m, int n);   /* declaration */

/* test power function */
main() {
    int i;
    for (i = 0; i < 10; ++i)
        printf("%d %d %d\n", i, power(2,i), power(-3,i)); /* calls */
    return 0;
}

/* power: raise base to n-th power; n >= 0 */
int power(int base, int n) {  /*definition */
    int i, p;
    p = 1;
    for (i = 1; i <= n; ++i)
        p = p * base;
    return p;
}
```

Function Definition Structure

A function definition has this form:

```
return-type function-name(parameter declarations, if any)
{
    declarations
    statements
}
```

- Function definitions can appear in any order.
- Parameters and variables in a function are local and not visible to other functions.

Function Call in `main`

The function `power` is called twice by `main`:

```
printf("%d %d %d\n", i, power(2,i), power(-3,i));
```

- Each call passes two arguments to `power`.
- `power` returns an integer to be formatted and printed.

Return Statement

- The return statement in a function specifies the value to be returned to the caller.
- A function may not return a value (useful for void functions).
- The calling function can ignore the returned value.
- `return(x);` and `return x;` are both valid.
- `return` statement could be omitted too; garbage returned.

Function Prototype

- Function prototype is another word for declaration.
- Must agree with the definition and uses of the function.

```
int power(int base, int n);
```

- Parameter names need not agree.
- Well-chosen names are good documentation.
- In ANSI C, function prototypes make declarations more explicit and error-resistant.

Arguments - Call by Value

- In C, all function arguments are passed "by value."
- This means that the called function is given the values of its arguments in temporary variables rather than the originals.
- Call by value is different from "call by reference" in languages like Fortran or with var parameters in Pascal.
- Call by value is an asset, leading to more compact programs with fewer extraneous variables.
- Parameters can be treated as conveniently initialized local variables in the called routine.
- To modify a variable in the calling routine, pointers are used.

Example: power(int base, int n)

```
/* power: raise base to n-th power; n >= 0; version 2 */
int power(int base, int n)
{
    int p;
    for (p = 1; n > 0; --n)
        p = p * base;
    return p;
}
```

- Parameter `n` is a temporary variable, counted down until it becomes zero.
- No need for an extra variable (`i`) in this version.

Arrays and Call by Value

- The story is different for arrays.
- When the name of an array is used as an argument, the value passed is the location or address of the beginning of the array.
- There is no copying of array elements.
- By subscripting this value, the function can access and alter any element of the array.

Character Arrays and Functions

- The most common type of array in C is the array of characters.
- Illustration: Write a program that reads text lines and prints the longest one.
- Divide the program into pieces: `getline`, `copy`, and `main`.
- `getline`: Reads a line into an array, returns the length.
- `copy`: Copies one array to another.
- `main`: Controls the process, finds and prints the longest line.

Character Arrays and Functions

```
#include <stdio.h>
#define MAXLINE 1000

int getline(char line[], int maxline);
void copy(char to[], char from[]);

int main(){
    int len;
    int max = 0;
    char line[MAXLINE];
    char longest[MAXLINE];

    while ((len = getline(line, MAXLINE)) > 0){
        if (len > max){
            max = len;
            copy(longest, line);
        }
    }
```

Character Arrays and Functions

```
    if (max > 0){
        printf("%s", longest);
    }

    return 0;
}
```

Character Arrays and Functions

```
int getline(char s[], int lim){
    int c, i;
    for (i=0; i < lim-1 && (c=getchar())!=EOF && c!='\n'; ++i)
        s[i] = c;

    if (c == '\n'){
        s[i] = c;
        ++i;
    }

    s[i] = '\0';
    return i;
}

void copy(char to[], char from[]){
    int i = 0;
    while ((to[i] = from[i]) != '\0')
        ++i;
}
```

Function Declarations and Parameters

- Functions `getline` and `copy` are declared at the beginning.
- Communication between `main` and `getline` is through parameters.
- `getline` returns the length of the line.

Null Character and String Representation

- Functions use the null character '\0' to mark the end of strings.
- In C, string constants are stored as arrays of characters with a '\0' termination.

Design Challenges

- Design problems arise even in small programs.
- Example: How to handle lines bigger than the limit in `main`?
- `getline` checks for overflow, but `copy` assumes the user knows the string sizes.

Example: Calculator Program

- Implementing a calculator in reverse Polish notation (Postfix expression).
- $(1 - 2) * (4 + 5)$ becomes $12 - 45 + *$.
- Use of a stack for operand manipulation.
- External variables for stack and its position.

```
while (next operator or operand is not EOF)
    if (number)
        push it
    else if (operator)
        pop operands, do operation, and push result
    else if (newline)
        pop and print top of stack
    else
        error
```

Example: Calculator Program

```
Consider the infix expression 3 + 4 * 11 - 8 + 9 / 3
The equivalent postfix expression is: 3 4 11 * + 8 - 9 3 / +
3    Push 3
4    Push 4
11   Push 11
*    Pop 11, Pop 4, Push 4*11=44
+    Pop 44, Pop 3, Push 3+44=47
8    Push 8
-    Pop 8, Pop 47, Push 47-8=39
9    Push 9
3    Push 3
/    Pop 3, Pop 9, Push 9/3=3
+    Pop 3, Pop 39, Push 39+3=42
\n   Pop and print 42
```